Clasificación de géneros de hongos mediante una red neuronal convolucional

Marco Iván Flores Villanueva
Tecnológico de Monterrey
A01276586@tec.mx

***Abstract*—Este documento presenta la implementación y evaluación de una red neuronal convolucional entrenada desde cero con TensorFlow y Keras para clasificar fotos de hongos en nueve géneros: Agaricus, Amanita, Boletus, Cortinarius, Entoloma, Hygrocybe, Lactarius, Russula y Suillus. El dataset usado contiene 6714 imágenes de distintos tamaños y presenta un desbalance de alrededor de cinco a uno entre la clase más grande y la más pequeña. A partir de un análisis exploratorio se descartó una imagen truncada, dos vacías y 68 duplicadas, quedando 6643 imágenes que se dividieron de manera estratificada en 60 % para entrenamiento, 20 % para validación y 20 % para prueba. Como primer modelo, se entrenó una CNN de tres bloques convolucionales durante 25 epochs, con el propósito de conseguir una línea base para diagnosticar su comportamiento. De igual forma, se eligió Macro-F1 como métrica principal, complementada con accuracy, precision, recall, F1-score por clase, matrices de confusión y cross-entropy. El modelo base obtuvo un Macro-F1 de 1.0000 en entrenamiento, 0.3396 en validación y 0.3408 en prueba, con una brecha de generalización de 0.6604, mientras que la pérdida de validación alcanzó su mínimo en la epoch 4 y después aumentó de 1.65 a 5.94. Estos resultados indican un sesgo bajo pero una varianza muy alta.**

***Keywords*—convolutional neural networks, image classification, mushrooms, TensorFlow, Keras, Macro-F1**

I. Introducción

Las redes neuronales convolucionales (CNN) son una arquitectura de deep learning diseñada para procesar datos con estructura de cuadrícula, como las imágenes. Una CNN aplica filtros que recorren la imagen para detectar patrones locales como texturas y formas, y al encimar varias capas convolucionales, es capaz de combinarlos en representaciones complejas. Esto le permite reconocer patrones sin importar la posición en la que aparezca en la imagen, algo que la vuelve muy útil para problemas de clasificación en imágenes.

Para este trabajo se usó el dataset Mushroom Classification Dataset [1], 6714 imágenes de hongos pertenecientes a nueve géneros comunes. En este dataset, cada observación es una fotografía tomada en condiciones naturales, con fondos, iluminación, ángulos y cantidad de ejemplares diferentes, y algunos de los géneros comparten formas o colores similares. Es por esto que el problema consiste en averiguar si una CNN puede aprender suficientes características visuales para clasificar los nueve géneros.

El objetivo de este trabajo es implementar modelos de deep learning mediante frameworks, evaluar sus desempeños sobre datos reales y diagnosticar sus niveles de sesgo, varianza y ajuste para mejorarlos después. Como primer modelo, se entrenó una CNN desde cero usando TensorFlow y Keras, sin técnicas de regularización, y sin transfer learning, para que su desempeño sirviera como línea base para medir el efecto de las mejoras en el siguiente modelo (pendiente). Posteriormente, el modelo base entrenado se guardó y se usó en otro programa que genera predicciones desde la consola.

II. Dataset y EDA

II-A. Descripción del dataset

El dataset se obtuvo de Kaggle [1] y contiene 6714 imágenes en formato JPG organizadas en nueve carpetas, una por cada género: Agaricus, Amanita, Boletus, Cortinarius, Entoloma, Hygrocybe, Lactarius, Russula y Suillus. El nombre de la carpeta funciona como la etiqueta de cada imagen, por lo que el dataset presenta un problema de clasificación multiclase con nueve clases. En la misma descripción del dataset se advierte que hay al menos una imagen truncada, por lo que se recomienda no cargarlo directamente sin una revisión previa.

En la Fig. 1 se muestran ejemplos aleatorios de cada género. Las imágenes incluyen uno o varios ejemplares, a veces cortados o vistos desde abajo, y sobre fondos como musgo, hojas o tierra, lo cual ocasiona una gran variabilidad dentro de una misma clase.

II-B. Revisión de la calidad de las imágenes

Antes de construir los modelos se recorrió cada archivo del dataset para revisar su estructura y calidad. Se verificó que todos los archivos fueran JPG, que pudieran abrirse, y se registró su tamaño, modo de color, brillo y contraste. Además, se calculó un hash md5 del contenido de cada archivo para detectar imágenes duplicadas aunque tuvieran nombres diferentes.

Todos los archivos tenían extensión JPG, pero una imagen de Russula no pudo abrirse por estar truncada, tal y como se advirtió en la página del dataset. De las 6713 imágenes restantes, 6711 se encontraban en modo RGB y 2 en escala de grises.

También se encontraron 138 imágenes que formaban 69 pares de duplicados, aunque ninguno de estos grupos contenía imágenes de clases distintas. Estas copias representan un riesgo de data leakage, pues una misma imagen podría terminar en el conjunto entrenamiento y en el de prueba al mismo tiempo. La mayoría se concentró en Lactarius, que contenía 58 de las copias.



Fig. 1. Ejemplos aleatorios de cuatro imágenes por género.

Tabla I
DISTRIBUCIÓN DE LAS IMÁGENES VÁLIDAS POR GÉNERO

Género	Imágenes	Porcentaje
Agaricus	353	5.26 %
Amanita	750	11.17 %
Boletus	1073	15.98 %
Cortinarius	836	12.45 %
Entoloma	364	5.42 %
Hygrocybe	316	4.71 %
Lactarius	1563	23.28 %
Russula	1147	17.09 %
Suillus	311	4.63 %
Total	**6713**	**100 %**

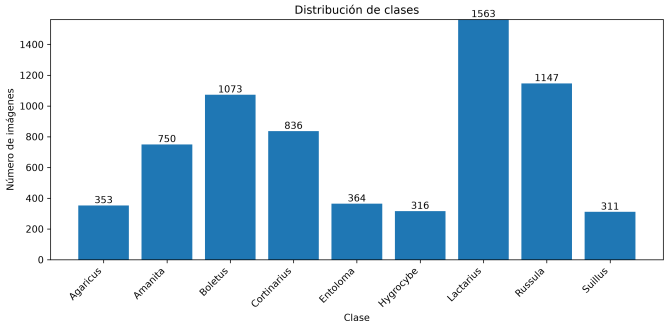


Fig. 2. Número de imágenes válidas por género.

Finalmente, el análisis de brillo mostró dos imágenes de Boletus con brillo de 1 y contraste de 0, las cuales son archivos completamente en blanco que no contienen ningún hongo, y por lo tanto no aportan información útil para el entrenamiento.

II-C. Distribución de clases

Como se observa en la Tabla I y en la Fig. 2, las 6713 imágenes válidas no se encuentran distribuidas de manera uniforme. Lactarius es la clase más grande, con 1563 imágenes (23.28 %), mientras que Suillus es la más pequeña, con 311 (4.63 %), lo que equivale a una razón de 5.03 entre ambas. Agaricus, Entoloma, Hygrocybe y Suillus cuentan cada una con menos del 6 % del dataset.

Este desbalance de clases se tomó en consideración durante la preparación y evaluación del modelo, pues un clasificador podría obtener un accuracy bueno acertando en las clases grandes y fallando en las pequeñas, por lo que se decidió hacer una partición estratificada de los datos, y se usó Macro-F1 para la evaluación.

II-D. Resolución y proporción de las imágenes

A diferencia de otros datasets de imágenes con tamaño fijo, las fotografías de este dataset presentan 1330 resoluciones distintas, y como se observa en la Fig. 3, la mayoría son horizontales. El ancho tiene una mediana de 800 píxeles y el alto de 580, con una proporción ancho/alto mediana de 1.34. Los valores extremos van desde 259 hasta 1280 píxeles de ancho, desde 152 hasta 1024 de alto y desde 0.56 hasta 2.86 en proporción.

Debido a que una CNN requiere entradas de un solo tamaño, todas las imágenes se tuvieron que redimensionar antes de entrar a la red a un tamaño de 150x150 píxeles. Se eligió este tamaño debido a que ninguna imagen mide menos de 150 píxeles en alguno de sus lados, por lo que el redimensionamiento siempre reduce la resolución y nunca las amplia. No obstante, al convertir una imagen con proporción de 1.34 en un cuadrado, su contenido se comprime horizontalmente respecto a la dirección vertical, resultando en una ligera deformación que se aceptó a cambio de conservar la imagen completa.

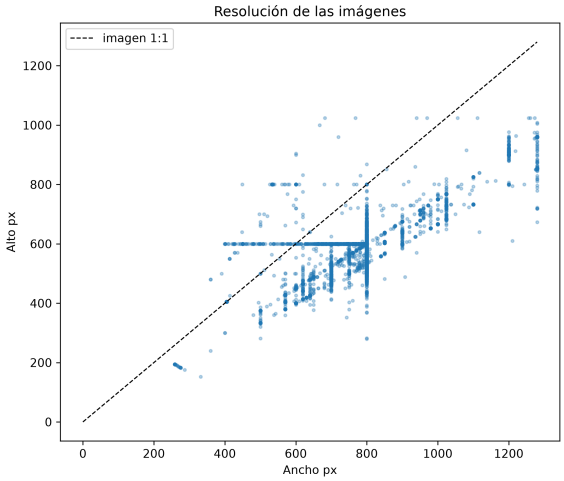


Fig. 3. Ancho y alto de cada imagen. La línea punteada corresponde a imágenes cuadradas.

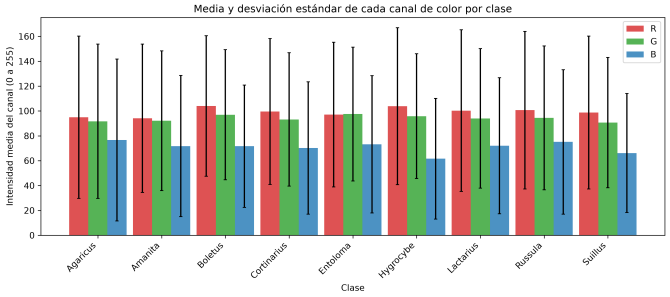


Fig. 4. Media y desviación estándar de cada canal RGB por género (escala de 0 a 255). La desviación se calcula sobre todos los píxeles de la clase.

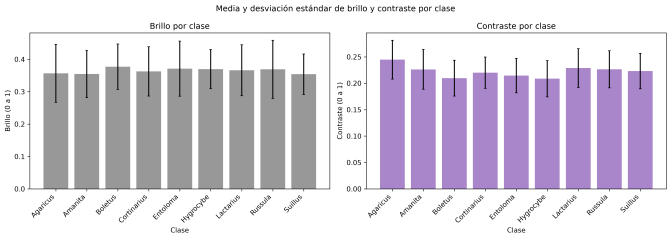


Fig. 5. Media y desviación estándar del brillo y del contraste por género. La desviación se calcula entre las imágenes de cada clase.

II-E. Color, brillo y contraste

Para determinar si los géneros podían distinguirse por propiedades globales de la imagen, se calculó la media y la desviación estándar de cada canal de color por clase (Fig. 4), así como el brillo y el contraste de cada imagen (Fig. 5). El brillo se definió como la media de la imagen en escala de grises y el contraste como su desviación estándar, ambos en una escala de 0 a 1. Para la estadística de color, cada imagen se redujo a 64x64 píxeles para que todas tuvieran el mismo peso en el promedio, sin importar su resolución.

Los resultados muestran que estas propiedades son muy similares entre los nueve géneros. La única diferencia relativamente notable fue que Hygrocybe presentó la media más baja del canal azul, lo que es consistente con los sombreros de colores cálidos característicos de este género. Aun así, generalmente el color, el brillo y el contraste globales no permiten separar las clases, por lo que el modelo necesita aprender características de forma y textura de los hongos.

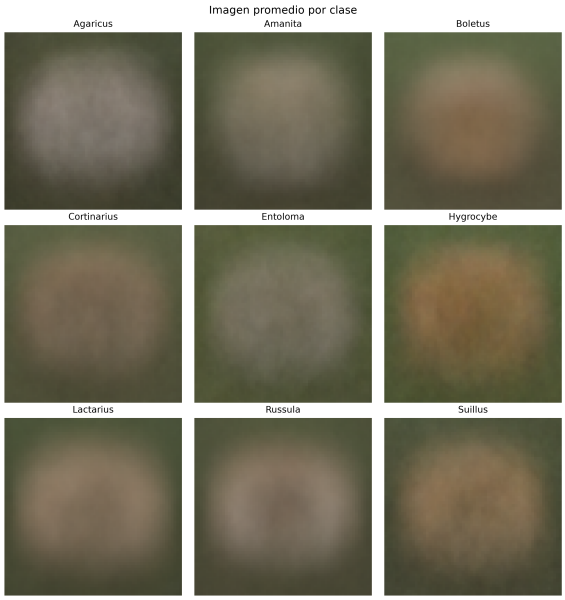


Fig. 6. Imagen promedio de cada género.

II-F. Imagen promedio por clase

Finalmente, se calculó la imagen promedio de cada género (Fig. 6). Estas imágenes promedio muestran manchas difusas de tonos similares entre las clases, lo cual ocurre porque los ejemplares aparecen en posiciones y tamaños diferentes en cada observación, y al promediarlas su forma se pierde. Este resultado consolida la idea de que la información de cada clase se encuentra en patrones locales que pueden aparecer en cualquier lugar, justamente el tipo de patrones que detectan las CNNs con sus filtros.

III. Preparación de datos y metodología de evaluación

III-A. Limpieza del dataset

A partir de la revisión de calidad se aplicaron tres pasos de limpieza antes de dividir los datos. Primero, se descartó la imagen truncada, ya que el generador de imágenes marcaría error al intentar leerla. Después, se descartaron las dos imágenes cuyos píxeles tenían todos el mismo valor (completamente blancas). Y por último, para los duplicados exactos se conservó únicamente la primera copia de cada grupo.

Como las dos imágenes blancas eran idénticas, formaban uno de los 69 grupos de duplicados, así que después de descartarlas quedaron 68 copias por eliminar. Como se resume en la Tabla II, el dataset final quedó formado por 6643 imágenes.

Tabla II
RESUMEN DE LA LIMPIEZA DEL DATASET

Concepto	Imágenes
Archivos originales	6714
Imágenes truncadas descartadas	1
Imágenes vacías descartadas	2
Copias duplicadas descartadas	68
Imágenes utilizadas	**6643**

Tabla III
NÚMERO DE IMÁGENES POR GÉNERO EN CADA CONJUNTO

Género	Train	Validation	Test	Total
Agaricus	210	70	71	351
Amanita	448	149	151	748
Boletus	642	214	214	1070
Cortinarius	500	166	168	834
Entoloma	218	72	74	364
Hygrocybe	189	63	63	315
Lactarius	903	301	301	1505
Russula	687	229	229	1145
Suillus	186	62	63	311
Total	**3983**	**1326**	**1334**	**6643**
Proporción	**59.96 %**	**19.96 %**	**20.08 %**	**100 %**

III-B. Separación en entrenamiento, validación y prueba

Debido al desbalance entre los géneros, la separación se realizó de manera estratificada y aleatoria clase por clase, usando una seed en concreto para que el proceso fuera reproducible. También, los archivos de cada clase se ordenaron alfabéticamente antes de mezclarlos para que la partición fuera igual sin importar el sistema operativo. Se asignó el 60 % de las imágenes de cada género al conjunto de entrenamiento, 20 % al de validación y 20 % al de prueba.

Posteriormente, las imágenes se copiaron a tres carpetas (train, validation y test), cada una con una subcarpeta por género. Así, el conjunto de prueba quedó separado desde el inicio del desarrollo. Después de la copia se verificó que ningún archivo apareciera en más de una partición. La Tabla III muestra el resultado de la separación.

Gracias a la estratificación, cada conjunto conservó aproximadamente la proporción original de cada género. Sin embargo, las clases pequeñas quedaron con entre 62-72 imágenes en validación, por lo que sus métricas individuales son más sensibles a pocos errores que las de las clases grandes.

III-C. Carga y preprocesamiento de las imágenes

Para cargar las imágenes se usó la clase ImageDataGenerator de Keras [3] junto con su método flow_from_directory. Este generador lee las imágenes por lotes directamente desde las carpetas, las convierte a RGB, las redimensiona a 150x150 píxeles y asigna a cada una la etiqueta de su carpeta como un entero de 0 a 8, en orden alfabético de los géneros. Además, los valores de los píxeles se escalaron del rango 0-255 al rango 0-1 mediante el parámetro rescale=1./255.

El generador de entrenamiento mezcla el orden de las imágenes en cada epoch, mientras que los generadores utilizados para evaluar no las mezclan, de forma que el orden de las predicciones coincida con el de las etiquetas reales al calcular las métricas.

III-D. Uso de los conjuntos en el desarrollo

La separación entre los tres conjuntos se mantuvo durante todo el desarrollo. El conjunto de entrenamiento se usó exclusivamente para el entrenamiento de la red. Después, el de validación se evaluó al final de cada epoch para observar el comportamiento y es el que se usó para diagnosticar el modelo y tomar decisiones sobre los ajustes. Y por último, el conjunto de prueba se reportó únicamente como referencia, y ninguna decisión sobre la arquitectura se tomó a partir de sus resultados.

III-E. Métricas de evaluación

Debido al desbalance de clases (por ejemplo, Lactarius representa el 22.7 % del conjunto de validación mientras que Suillus sólo el 4.7 %), se seleccionó Macro-F1 como métrica principal. Esta métrica calcula el F1-score de cada género por separado y luego obtiene su promedio, por lo que las nueve clases tienen el mismo peso y el resultado global no queda influenciado por las clases con más imágenes. El F1-score de cada clase c corresponde a la media armónica de su precision P_c y su recall R_c:

$$F1_c = \frac{2\,P_c\,R_c}{P_c + R_c}, \qquad \text{Macro-F1} = \frac{1}{K}\sum_{c=1}^{K} F1_c, \tag{1}$$

donde $K = 9$ es el número de géneros.

Como métricas complementarias se usaron accuracy, precision, recall y el F1-score individual de cada clase, calculados con scikit-learn [4], además de matrices de confusión para identificar qué géneros se confunden entre sí. También se registró la función de pérdida en entrenamiento y validación durante cada epoch para que sus curvas permitieran detectar underfitting u overfitting.

III-F. Criterios para diagnosticar sesgo, varianza y ajuste

Además del desempeño del modelo, se compararon los resultados de entrenamiento y validación para analizar su sesgo, varianza y nivel de ajuste. Un desempeño bajo tanto en train como en validation se interpretó como señal de underfitting y sesgo alto, mientras que un desempeño alto en train pero bajo en validation se interpretó como una señal de varianza alta y overfitting. Para cuantificar esta diferencia se calculó la brecha de generalización:

$$\text{Gap} = \text{Macro-F1}_{\text{train}} - \text{Macro-F1}_{\text{val}}. \tag{2}$$

Esta brecha no se consideró como un indicador definitivo, pues también se tomaron en cuenta las métricas por clase, las matrices de confusión y las curvas de pérdida.

Tabla IV
ARQUITECTURA DE LA CNN BASE

Capa	Salida	Parámetros
Conv2D (32, 3x3, ReLU)	150x150x32	896
MaxPooling2D (2x2)	75x75x32	0
Conv2D (64, 3x3, ReLU)	75x75x64	18,496
MaxPooling2D (2x2)	37x37x64	0
Conv2D (128, 3x3, ReLU)	37x37x128	73,856
MaxPooling2D (2x2)	18x18x128	0
Flatten	41,472	0
Dense (128, ReLU)	128	5,308,544
Dense (9, softmax)	9	1,161
Total		**5,402,953**

Tabla V
CONFIGURACIÓN DEL ENTRENAMIENTO

Parámetro	Valor
Tamaño de entrada	150x150x3
Optimizador	Adam (valores por defecto)
Función de pérdida	Sparse categorical cross-entropy
Tamaño de lote	32
Lotes por epoch	125
Epochs	25
Data augmentation	No
Dropout	No
Early stopping	No
Seed	67

IV. IMPLEMENTACIÓN DEL MODELO BASE

IV-A. Framework y estructura del código

El modelo se implementó en Python con TensorFlow y su API de alto nivel Keras [2], y el entrenamiento se realizó en el CPU de una computadora MacBook Air M1. La solución se dividió en programas independientes: eda.py para el análisis exploratorio, data_setup.py para la limpieza y partición de los datos, train_base.py para el entrenamiento y la evaluación, y predict.py para hacer predicciones con el modelo guardado.

IV-B. Arquitectura

La red se construyó como un modelo Sequential formado por tres bloques convolucionales y un clasificador denso, como se muestra en la Tabla IV. Cada bloque contiene una capa Conv2D con filtros de 3x3, padding same y activación ReLU, seguida de una capa MaxPooling2D de 2x2 que reduce a la mitad las dimensiones espaciales. El número de filtros aumenta de 32 a 64 y a 128, para que así los primeros bloques detecten patrones simples y los siguientes combinaciones más complejas de ellos.

Después del último bloque la capa Flatten convierte los mapas de características de 18x18x128 en un vector de 41,472 valores, que pasa por una capa densa de 128 neuronas y finalmente por una capa de salida de 9 neuronas con activación softmax. Esta última transforma las salidas z_k de la red en una probabilidad para cada género:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \ldots, K, \tag{3}$$

y la clase predicha corresponde a la de mayor probabilidad.

El modelo tiene un total de 5,402,953 de parámetros entrenables. Sin embargo, la Tabla IV muestra que el 98.25 % de ellos se encuentra en la primera capa densa, ya que cada uno de los 41,472 valores del vector aplanado se conecta con cada una de sus 128 neuronas. Esto significa alrededor de 1357 parámetros por cada imagen de entrenamiento, una proporción que anticipa un riesgo de overfitting.

IV-C. Función de pérdida y configuración del entrenamiento

Como las etiquetas se representan como enteros de 0 a 8, se usó como función de pérdida la entropía cruzada categórica en su versión sparse:

$$L = -\frac{1}{n} \sum_{i=1}^{n} \log \hat{y}_{i,y_i}, \tag{4}$$

donde n es el número de imágenes, y_i la clase real de la imagen i y $\hat{y}_{i,y_i}$ la probabilidad que el modelo asigna a esa clase. Esta función penaliza más cuanto menor sea la probabilidad asignada a la clase correcta, por lo que un error cometido con mucha confianza produce una pérdida mucho mayor que uno cometido con poca confianza.

La red se optimizó con Adam [5] usando sus valores por defecto, con lotes de 32 imágenes y durante 25 epochs, como se resume en la Tabla V. Este primer modelo no incluyó data augmentation, dropout ni early stopping. El número de epochs se mantuvo fijo para que las curvas mostraran el momento en el que el desempeño de validación deja de mejorar, y así poder diagnosticar el comportamiento del modelo. Cada entrenamiento tomó aproximadamente 18 minutos, con entre 35 y 59 segundos por epoch.

V. RESULTADOS Y DIAGNÓSTICO DEL MODELO BASE

V-A. Curvas de entrenamiento

La Fig. 7 muestra la evolución de la accuracy y de la pérdida en entrenamiento y validación. Durante las primeras cuatro epochs ambas curvas avanzaron casi igual, y es en la epoch 4 donde se obtuvo tanto la menor pérdida como la mayor accuracy de validación de todo el entrenamiento.

No obstante, a partir de la epoch 5 las curvas se separaron abruptamente. Este comportamiento muestra que después de ese punto, el modelo dejó de aprender patrones generalizables y comenzó a memorizar las imágenes de entrenamiento. Es importante destacar que la pérdida de validación siguió aumentando aunque su accuracy se mantuvo casi constante. Como la accuracy no cambió, el modelo cometió aproximadamente la misma cantidad de errores, sin embargo, según (4), la pérdida crece cuando disminuye la probabilidad asignada a la clase correcta, por lo que su aumento indica que esos errores se cometieron cada vez con mayor confianza, asignando a la clase correcta probabilidades cercanas a cero.

V-B. Resultados en entrenamiento y validación

Al terminar las 25 epochs, el modelo base fue evaluado sobre los conjuntos de train y validation, obteniendo los resultados de la Tabla VI.

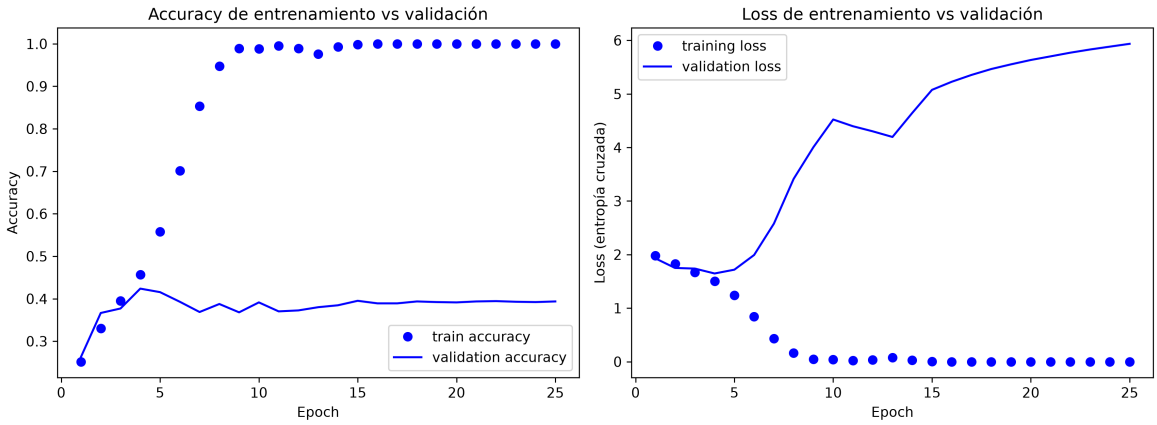


Fig. 7. Accuracy (izquierda) y pérdida (derecha) en entrenamiento y validación durante las 25 epochs.

Tabla VI
RESULTADOS DEL MODELO BASE AL FINAL DEL ENTRENAMIENTO

Conjunto	Macro-F1	Accuracy	Pérdida
Train	1.0000	1.0000	0.0001
Validation	0.3396	0.3937	5.9358
Generalization gap (Macro-F1): 0.6604			

Tabla VII
MÉTRICAS POR GÉNERO EN VALIDACIÓN

Género	Precision	Recall	F1-score	Imágenes
Agaricus	0.1935	0.1714	0.1818	70
Amanita	0.3725	0.3826	0.3775	149
Boletus	0.6093	0.6121	0.6107	214
Cortinarius	0.2222	0.2169	0.2195	166
Entoloma	0.3617	0.2361	0.2857	72
Hygrocybe	0.5676	0.3333	0.4200	63
Lactarius	0.4204	0.4651	0.4416	301
Russula	0.3723	0.4454	0.4056	229
Suillus	0.1395	0.0968	0.1143	62

El modelo clasificó correctamente las 3983 imágenes de entrenamiento, pero en validación sólo 522 de 1326, con una accuracy de 0.3937 y un Macro-F1 de 0.3396. Aunque este desempeño es bajo, su accuracy de validación es mayor que la que se esperaría eligiendo uno de los nueve géneros al azar (1/9 = 0.1111), por lo que sí aprendió algunos patrones útiles para distinguirlos, aunque la mayor parte de lo aprendido no se generalizó.

La Tabla VII muestra que el desempeño fue muy disparejo entre los géneros. Boletus obtuvo el mayor F1-score, seguido por Lactarius y Hygrocybe. Por otro lado, los géneros Suillus y Agaricus obtuvieron F1-scores muy bajos, destacando que ambas clases tienen menos de 71 imágenes en validación. Se observa que los géneros con menos imágenes obtuvieron un recall menor, pues el modelo tendió a predecir con más frecuencia las clases grandes.

V-C. Matriz de confusión de validación

La matriz de confusión de la Fig. 8 muestra que la distribución de los 804 errores en validación no fue uniforme. La

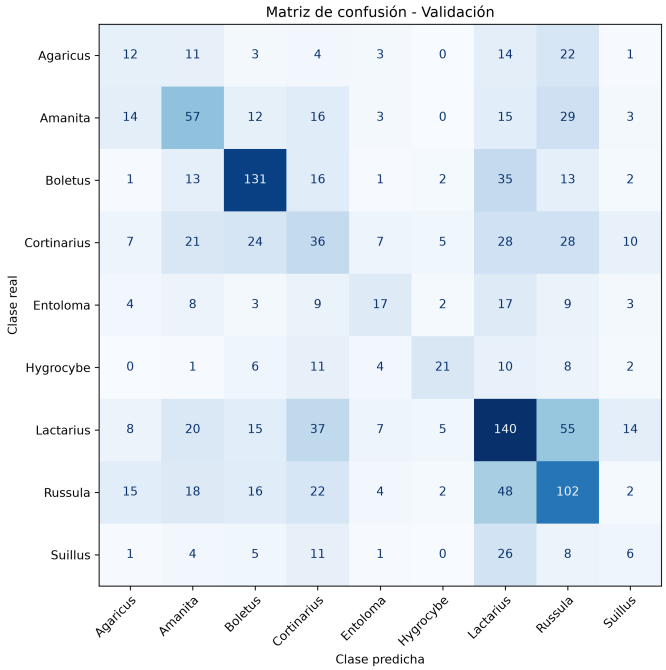


Fig. 8. Matriz de confusión del modelo base en validación.

confusión más frecuente fue entre Lactarius y Russula, lo cual podría deberse a que ambos géneros pertenecen a la misma familia de hongos y tienen una apariencia similar.

Otras confusiones frecuentes fueron Lactarius clasificado como Cortinarius, Boletus como Lactarius y Amanita como Russula. Un caso interesante es Suillus, pues 26 de sus 62 imágenes de validación se clasificaron como Lactarius, más que las 6 que se clasificaron correctamente. Esto confirma que el modelo usa a las clases grandes como predicción por default cuando no reconoce con claridad una imagen.

V-D. Evaluación en prueba

Como referencia, el modelo también se evaluó sobre el conjunto de prueba, obteniendo los resultados de la Tabla VIII.



Fig. 9. Predicciones del modelo base sobre ocho imágenes al azar del conjunto de prueba. En verde se muestran los aciertos y en rojo los errores, junto con la confianza de cada predicción.

Tabla VIII
COMPARACIÓN ENTRE VALIDACIÓN Y PRUEBA

Conjunto	Macro-F1	Accuracy	Pérdida
Validation	0.3396	0.3937	5.9358
Test	0.3408	0.3943	6.1990

Tabla IX
DIAGNÓSTICO DEL MODELO BASE

Modelo	Sesgo	Varianza	Nivel de ajuste
CNN base	Bajo	Muy alta	Overfitting grave

Los resultados de prueba y validación son casi iguales, lo que indica que el conjunto de validación estima de forma confiable el desempeño del modelo sobre imágenes que no vio durante el entrenamiento.

V-E. Diagnóstico de sesgo, varianza y nivel de ajuste

El Macro-F1 perfecto en training indica que el modelo tiene capacidad de sobra para representar los datos de entrenamiento, por lo que no muestra señales de underfitting y su sesgo es bajo. Sin embargo, la brecha de generalización de 0.6604, el resultado de 0.3396 en validation y su aumento en pérdida después de la epoch 4 indican una varianza muy alta.

Como se resume en la Tabla IX, se diagnostica que la CNN base presenta un overfitting grave. Las principales causas identificadas son la gran cantidad de parámetros de la primera capa densa en relación con las 3983 imágenes de entrenamiento, la ausencia de técnicas de regularización, el entrenamiento que fue más allá del punto en el que la validación dejó de mejorar, y la dificultad del propio problema identificada en el EDA.

VI. PREDICCIONES

Para generar predicciones con el modelo entrenado se implementó el programa predict.py, que carga el modelo guardado, aplica a ocho imágenes el mismo preprocesamiento del entrenamiento, y predice sus géneros. La Fig. 9 muestra un ejemplo de la figura resultante, donde el modelo acertó cuatro de ocho imágenes, un resultado consistente con el accuracy de 0.39 en prueba y para una muestra pequeña como ocho imágenes.

Cabe mencionar que este programa será reemplazado por una interfaz próximamente.

Lo más interesante de las predicciones es la confianza con la que el modelo se equivoca, pues las cuatro predicciones incorrectas tuvieron una confianza de casi 100 %, mientras que 2 de los aciertos tuvieron confianzas de apenas 57.90 % y 59.50 %. Este comportamiento concuerda con el aumento de la pérdida de validación observado en las curvas, ya que al llevar la pérdida de entrenamiento casi a cero, el modelo aumentó la magnitud de sus salidas hasta producir probabilidades extremas, y como gran parte de las características que aprendió son específicas de las imágenes de entrenamiento, al fallar sobre imágenes nuevas lo hace con una confianza muy alta. Esta mala calibración es otro síntoma del overfitting.

REFERENCIAS

[1] "Mushroom Classification Dataset ," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/lizhecheng/mushroom-classification
[2] F. Chollet, *Deep Learning with Python*. Shelter Island, NY, USA: Manning, 2018.
[3] "tf.keras.preprocessing.image.ImageDataGenerator," TensorFlow API documentation. [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image/ImageDataGenerator

[4] “sklearn.metrics.classification_report,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html
[5] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015, arXiv:1412.6980.